

Молдавский Государственный Университет
Факультет Математики и Информатики
Департамент Информатики

**Индивидуальная работа
По предмету «РНР»**

Выполнил: Майбуров Артём
Группа: IA-2404

Кишинёв 2026

СОДЕРЖАНИЕ

1. **Теоретическая часть**
 - 1.1. Веб-архитектура MVC (Model-View-Controller)
 - 1.2. Безопасность веб-приложений и хэширование данных
2. **Формулировка задачи**
3. **Цель и основные этапы работы**
4. **Практическая часть**
 - 4.1. Особенности реализации приложения
 - 4.2. Функциональные возможности компонентов
 - 4.3. Сценарии взаимодействия пользователей с приложением (Use Cases)
 - 4.4. Структура базы данных (Реляционная модель)
5. **Выводы и ссылка на репозиторий.**

1. Теоретическая часть

1.1. Веб-архитектура MVC (Model-View-Controller)

Архитектурный паттерн MVC является международным стандартом промышленной веб-разработки. Его применение направлено на разделение бизнес-логики приложения от интерфейса пользователя, что минимизирует связанность компонентов (coupling) и повышает гибкость системы.

- **Model (Модель):** Слой, инкапсулирующий данные приложения и методы работы с ними (в данном проекте реализован через прямые SQL-запросы посредством безопасного драйвера PDO). Модель полностью изолирована от визуальной составляющей.
- **View (Представление):** Слой, отвечающий исключительно за формирование выходного HTML-кода на основе переданных ему переменных. Шаблоны не содержат логики обработки запросов или прямого обращения к серверу БД.
- **Controller (Контроллер):** Связующее звено. Контроллер перехватывает HTTP-запросы пользователя, валидирует входные параметры, обращается к Модели за данными и передает результат в соответствующее Представление.

1.2. Безопасность веб-приложений и хэширование данных

Обеспечение безопасности современных многопользовательских веб-сервисов строится на трех фундаментальных принципах:

1. **Защита от XSS (Cross-Site Scripting):** Достигается обязательным экранированием любого динамического вывода на стороне клиента. Применение функции htmlspecialchars() с флагом ENT_QUOTES нейтрализует вредоносные JavaScript-инъекции, превращая спецсимволы в безопасные HTML-сущности.
2. **Безопасное хэширование паролей:** Хранение паролей в открытом виде категорически запрещено. В проекте задействован криптографический стандарт bcrypt через встроенную функцию PHP password_hash(..., PASSWORD_DEFAULT). Она генерирует уникальную 60-символьную строку, устойчивую к атакам по словарям и методу "грубой силы" (Brute Force). Сверка паролей при аутентификации выполняется через безопасную функцию password_verify().
3. **Изоляция сессий:** Управление доступом к защищенным компонентам реализовано через суперглобальный массив \$_SESSION. Флаг session.cookie_httponly = 1 предотвращает кражу сессионных куки-файлов через клиентские скрипты, блокируя атаки типа Session Hijacking.

2. Формулировка задачи

Наименование проекта: Веб-портал аномальных явлений и странных фактов «StrangePortal».

Постановка задачи: Разработать модульное многопользовательское веб-приложение средней сложности на базе реляционной СУБД MySQL. Система должна разграничивать права доступа на основе ролей пользователей (*Гость, Авторизованный пользователь, Администратор*). Приложение обязано содержать общедоступный динамический контент, сквозную систему поиска, интерактивную форму генерации контента с комплексной двухфакторной валидацией (Client-side / Server-side), а также скрытую панель администратора с возможностью полного CRUD-управления базой данных и масштабирования административного состава.

3. Цель и основные этапы работы

Цель работы: Проектирование, программная реализация и развертывание защищенного веб-приложения с использованием объектно-ориентированного подхода, архитектурных шаблонов проектирования и реляционных баз данных.

Основные этапы работы:

1. Проектирование модели данных: Разработка схемы реляционной базы данных, определение типов полей, ограничений и внешних связей.
2. Разработка ядра и конфигурации: Настройка безопасного PDO-соединения с БД, конфигурация параметров сессий.
3. Реализация слоя логики (Controllers): Написание контроллера авторизации (AuthController), главного контроллера (MainController) и защищенного контроллера управления (AdminController).
4. Верстка и интеграция представлений (Views): Создание общего макета страницы (layout.php) и динамически подгружаемых компонентов форм и таблиц.
5. Внедрение систем безопасности: Программирование модулей валидации входных данных, экранирования вывода и хэширования учетных данных.
6. Тестирование и деплой: Проверка граничных сценариев взаимодействия, наполнение БД тестовыми записями, выгрузка проекта в удаленный репозиторий GitHub.

3. Практическая часть

4.1. Особенности реализации приложения

Приложение построено на базе архитектуры Front Controller. Единственной физической точкой входа является файл index.php, который выполняет функции роутера (маршрутизатора). На основе GET-параметра page роутер инициализирует соответствующий класс-контроллер.

Связь с базой данных MySQL инкапсулирована внутри функции getDBConnection(). Параметры PDO настроены на явный запрет эмуляции подготовленных выражений (PDO::ATTR_EMULATE_PREPARES => false), что на корню ликвидирует возможность проведения SQL-инъекций при передаче параметров в запросы.

4.2. Функциональные возможности компонентов

Веб-приложение строго разделено на функциональные зоны в соответствии с техническим заданием:

- **Общедоступный компонент:** Доступен неавторизованным пользователям. Включает в себя динамический вывод подтвержденных фактов из БД (таблица обновляется автоматически при одобрении записей администратором) и форму поиска.
- **Формы взаимодействия:**
 - Форма поиска (Форма 1): Фильтрует архив записей на лету по совпадениям в заголовках или описании с помощью SQL-оператора LIKE.
 - Форма создания ресурса (Форма 2): Содержит 5 разнотипных полей (text, select/enum, date, number, textarea). Снабжена клиентскими ограничениями (required, minlength) и дублирующей строгой серверной проверкой на PHP.
- **Защищенный компонент (Панель администратора):** Доступен пользователям с флагом role = 'admin'. Содержит 4 встроенные функции администрирования:

1. Просмотр всех существующих записей в БД (включая немодерированные).
2. Одобрение публикации новых ресурсов (Перевод флага `is_verified` из 0 в 1).
3. Полное удаление записей из базы данных (Операция DELETE).
4. Создание новых учетных записей с административными правами доступа из интерфейса панели.

4.3. Сценарии взаимодействия пользователей с приложением

Сценарий А: Гость (Неавторизованный пользователь)

1. Пользователь заходит на главную страницу приложения `index.php`.
2. Система отправляет запрос к БД и динамически выводит таблицу проверенных фактов.
3. Пользователь использует форму поиска: вводит ключевое слово, отправляет форму, страница обновляется, отображая отфильтрованный контент.
4. Пользователь заполняет форму создания ресурса и отправляет её. Данные проходят валидацию. Запись сохраняется в БД со статусом `is_verified = 0` (не отображается на главной). Пользователь видит сообщение об успешной отправке на модерацию.

Сценарий Б: Администратор системы

1. Администратор переходит на страницу `index.php?page=auth` и вводит свои учетные данные.
2. Система проверяет хэш пароля. При успехе в сессию записывается `$_SESSION['role'] = 'admin'`, в шапке сайта появляется скрытая ссылка на Панель Администратора.
3. Администратор переходит по ссылке `index.php?page=admin`.
4. Администратор видит таблицу модерации. Напротив предложенного Гостем факта он нажимает «Одобрить». Статус записи меняется в БД.
5. Администратор заполняет форму добавления нового администратора. В базу данных заносится новый пользователь с хэшированным паролем и ролью `admin`.

4.4. Структура базы данных (Реляционная модель)

Для обеспечения работы приложения развернута реляционная база данных `strange_portal`, состоящая из двух таблиц.

Таблица 1: users (Учетные записи пользователей)

Поле	Тип данных	Ограничения	Описание
id	INT	AUTO_INCREMENT, PRIMARY KEY	Уникальный идентификатор
username	VARCHAR(50)	NOT NULL, UNIQUE	Уникальное имя (логин)
password_hash	VARCHAR(255)	NOT NULL	Безопасный хэш пароля (bcrypt)

Поле	Тип данных	Ограничения	Описание
role	ENUM('user', 'admin')	NOT NULL, DEFAULT 'user'	Роль в системе управления
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата и время регистрации

Таблица 2: facts (Ресурсы / Странные факты)

Поле	Тип данных	Ограничения	Описание
id	INT	AUTO_INCREMENT, PRIMARY KEY	Уникальный идентификатор
title	VARCHAR(100)	NOT NULL	Заголовок ресурса (Поле типа String)
category	ENUM('Космос', 'Природа', 'Наука', 'История')	NOT NULL	Категория (Поле типа Enum)
discovery_date	DATE	NOT NULL	Дата фиксации (Поле типа Date)
weirdness_level	INT	NOT NULL	Числовой уровень от 1 до 10
description	TEXT	NOT NULL	Длинный текст описания (Тип Text)
is_verified	TINYINT(1)	NOT NULL, DEFAULT 0	Флаг модерации (0 - скрыт, 1 - активен)

Поле	Тип данных	Ограничения	Описание
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Системная дата добавления в

ВЫВОДЫ

В ходе выполнения выпускной лабораторной работы были успешно решены все поставленные задачи и достигнута главная цель — проектирование и программная реализация защищенного многопользовательского веб-приложения «StrangePortal» с использованием объектно-ориентированного подхода и реляционной базы данных MySQL. На основе проделанной работы можно сделать следующие ключевые выводы:

1. **Эффективность архитектуры MVC:** Разделение приложения на независимые слои (Модели, Представления и Контроллеры) полностью оправдало себя на практике. Использование единой точки входа (index.php) позволило централизовать маршрутизацию (роутинг), упростить контроль сессий и исключить появление дублирующего кода. Такая структура делает проект легко масштабируемым: добавление новых страниц или изменение дизайна не затрагивает существующую бизнес-логику.
2. **Надежность реляционных СУБД:** Интеграция базы данных MySQL через встроенный интерфейс PDO обеспечила высокую скорость и стабильность работы с динамическим контентом. Использование механизма подготовленных выражений (Prepared Statements) с полным отключением эмуляции гарантировало абсолютную защищенность системы от одного из самых опасных типов уязвимостей — SQL-инъекций.
3. **Обеспечение комплексной безопасности:** Внедрение криптографического стандарта *bcrypt* для хэширования паролей позволило защитить учетные данные пользователей от компрометации даже в случае потенциальной утечки базы данных. Двухуровневая валидация данных (на стороне клиента средствами HTML5 и на стороне сервера средствами PHP), совмещенная с обязательным экранированием через htmlspecialchars(), обеспечила надежную защиту от межсайтового скриптинга (XSS) и гарантировала целостность хранящейся информации.
4. **Разграничение прав доступа:** Реализованный механизм сессий позволил гибко и безопасно разграничить зоны видимости сайта для разных типов пользователей. Изолированный защищенный компонент (Панель администратора) полностью выполняет свои функции управления базой данных (CRUD) и масштабирования штата модераторов, оставаясь при этом абсолютно недоступным для рядовых пользователей и злоумышленников.

Ссылка на репозиторий: <https://github.com/Artiom-Maiburov/phplabs>